

# SDV Fault-Prediction & Self-Healing — Full Tool Stack (0 → N)

Working title: *AI solutions for Fault Prediction and Self-Healing inner & cross-ECU Software-Defined Vehicle architecture* Scope here: every tool, layer by layer, mapped to your six figures. 1-year build, solo.

## Legend

- HAVE** **HAVE** — already on your CV / in your projects. Don't relearn, just point it at this problem.
- LEARN** **LEARN** — new, and on the critical path. Budget real time.
- OPT** **OPTIONAL** — stretch / nice-to-have. Skip for v1 unless time allows.

## The minimum viable stack (install these first)

If you only set up twelve things to get a working closed loop, it's these. Everything else in this doc is depth you add once the spine works.

- Obsidian** (+ Git) — research brain & thesis notes **HAVE**-ish
- Python 3.13 + uv** — ML environment **LEARN**
- VS Code + Jupyter** — experiments **HAVE**
- PyTorch** — train the autoencoder & GNN **LEARN**
- scikit-learn** — baselines + metrics **LEARN**
- Renode** — simulate the multi-ECU network, inject faults **LEARN**
- SocketCAN / vcan** — virtual CAN FD bus for telemetry **LEARN**
- LiteRT for Microcontrollers** (ex-TFLite Micro) — on-MCU inference **LEARN**
- NXP eIQ Toolkit + eIQ Auto ML SDK** — deploy to Neutron NPU **HAVE** (insider access)
- FastAPI + Postgres/pgvector** — off-vehicle backbone **HAVE**
- Grafana** — telemetry dashboard **LEARN** (easy)
- Docker + Railway** — run it all **HAVE**

## Layer 0 — Knowledge, research & thesis workflow

This is where **Obsidian** lives. For a 1-year thesis the bottleneck isn't usually code — it's not losing the thread across papers, experiments, and decisions. Obsidian is your second brain for exactly that.

| Tool                                                   | Role                                                                                                                               | Status       |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|--------------|
| <b>Obsidian</b>                                        | Research vault: literature notes, concept maps (MOCs), daily logs, architecture decision records (ADRs), the thesis outline itself | <b>HAVE</b>  |
| <b>Zotero</b> + Better BibTeX                          | Reference manager — collect every paper, auto-generate citations                                                                   | <b>LEARN</b> |
| Obsidian ↔ Zotero plugin ( <b>zotero-integration</b> ) | Pull paper metadata + annotations into notes                                                                                       | <b>LEARN</b> |
| Obsidian <b>Excalidraw</b> plugin                      | Redraw your six figures as editable diagrams inside notes                                                                          | <b>OPT</b>   |
| Obsidian <b>Dataview</b> plugin                        | Query your own notes (e.g. "all faults I've defined", "all open questions")                                                        | <b>OPT</b>   |
| Obsidian <b>Kanban</b> plugin                          | Track the 1-year milestones as a board                                                                                             | <b>OPT</b>   |
| Obsidian <b>Git</b> plugin                             | Version + back up the vault automatically                                                                                          | <b>LEARN</b> |
| <b>Overleaf / LaTeX</b> (or <b>Typst</b> )             | The actual thesis + IEEE/SAE paper. PhD-grade docs are almost always LaTeX; Typst is the faster modern alternative                 | <b>LEARN</b> |
| <b>draw.io</b> / your figure tool                      | Polished architecture diagrams for the thesis                                                                                      | <b>HAVE</b>  |

### A concrete Obsidian workflow for this project

Structure the vault so it mirrors the architecture, not a generic notes dump:

/00-MOCs/  
/10-Literature/  
/20-Concepts/  
/30-Architecture/  
/40-Experiments/  
/50-Thesis/  
/99-Daily/

+ maps of content: "Anomaly Detection MOC", "GNN MOC", "Safety MOC"  
+ one note per paper (Zotero-linked), your summary in your words  
+ atomic notes: "denoising autoencoder", "ASIL-D", "fault injection"  
+ one note per figure; ADRs ("why deterministic supervisor, not ML")  
+ one note per experiment: hypothesis, config, result, link to W&B run  
+ chapter drafts that later export to LaTeX  
+ daily log: what you tried, what broke, what's next

The payoff: when you write the thesis, every claim already links back to a literature note and an experiment note. No "where did I read that?" archaeology.

## Layer 1 — Core dev environment & languages

| Tool                                       | Role                                                                 | Status |
|--------------------------------------------|----------------------------------------------------------------------|--------|
| Python 3.13                                | The ML language. This is the big new daily driver                    | LEARN  |
| uv                                         | Modern, fast Python package + venv manager (replaces pip/conda pain) | LEARN  |
| C / Embedded C                             | On-ECU runtime, deterministic supervisor                             | HAVE   |
| AArch64 / Cortex-M asm                     | Low-level glue, timing-critical paths                                | HAVE   |
| VS Code + Python/Jupyter/clangd extensions | One editor for ML + firmware                                         | HAVE   |
| Jupyter notebooks                          | Rapid ML experimentation                                             | LEARN  |
| Git / GitHub                               | Version control                                                      | HAVE   |
| Docker                                     | Reproducible environments                                            | HAVE   |
| WSL2 (if on Windows)                       | Linux for SocketCAN, Renode, ML                                      | LEARN  |

## Layer 2 — ML / model development (your real gap)

This layer maps directly onto the boxes in **Figure 3 & 4**. Each model family = a tool.

| Tool                           | Maps to                      | Role                                                                               | Status        |
|--------------------------------|------------------------------|------------------------------------------------------------------------------------|---------------|
| PyTorch                        | DETECT, PREDICT              | Primary deep-learning framework (2026 default)                                     | LEARN         |
| scikit-learn                   | DETECT baseline              | Isolation Forest, One-Class SVM, metrics, train/test split                         | LEARN         |
| pandas / NumPy / polars        | all                          | Data wrangling for telemetry tables                                                | LEARN         |
| Autoencoder (in PyTorch/Keras) | <b>DETECT</b> (Analyst)      | Denoising autoencoder → reconstruction-error anomaly score                         | LEARN         |
| PyTorch Geometric (PyG)        | <b>PREDICT</b> (Predictor)   | Graph Neural Networks for fault propagation across the ECU topology                | LEARN         |
| DoWhy / causal-learn (PyWhy)   | <b>ROOT-CAUSE</b>            | Causal graphs, root-cause inference                                                | OPT           |
| Stable-Baselines3 + Gymnasium  | <b>ACT</b> (recovery policy) | Reinforcement learning for recovery-policy search                                  | OPT (stretch) |
| Weights & Biases or MLflow     | all                          | Experiment tracking + model registry — non-negotiable once you have >5 experiments | LEARN         |
| Optuna                         | all                          | Hyperparameter search                                                              | OPT           |
| SHAP / Captum                  | Reporter / safety            | Explainability (XAI) — feeds the audit/Reporter box, helps the ISO-26262 story     | OPT           |
| matplotlib / seaborn / plotly  | all                          | Analysis plots                                                                     | LEARN         |

**Note on PyTorch vs TensorFlow:** train in PyTorch (better ergonomics, dominant in research), then convert to `.tflite` for deployment via **ai-edge-torch**. You get research-friendly training *and* the NXP-friendly deployment format. Don't split your effort across both frameworks for training.

## Layer 3 — Data: telemetry generation & fault injection

This solves the document's #1 risk — *"fault data is a secret industrial asset, hard to get."* You **generate** it.

| Tool                                                                                  | Role                                                                                                                                                                 | Status |
|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Renode                                                                                | Simulate S32K3 / S32N / S32K5 as a multi-node network; run unmodified firmware; deterministic fault injection (corrupt a register, drop a bus frame, spike CPU load) | LEARN  |
| Linux SocketCAN + vcan                                                                | Free virtual CAN FD bus — generate "CAN error frames" telemetry from your figures                                                                                    | LEARN  |
| Custom fault-injection scripts                                                        | Define the fault taxonomy (timing jitter, memory leak, thermal drift, packet loss) and inject on a schedule                                                          | LEARN  |
| Public datasets (C-MAPSS turbofan, bearing-fault sets, automotive CAN intrusion sets) | Practice anomaly/RUL models before your own data exists                                                                                                              | OPT    |
| CARLA                                                                                 | Full vehicle/sensor simulator — only if you need driving scenarios. Likely overkill for v1                                                                           | OPT    |

**Strategy:** Renode + SocketCAN gives you a reproducible "vehicle" on your laptop. You inject a known fault, log the telemetry, and now you have *labeled* fault data — the thing the document says is impossible to get. That reproducibility is itself a thesis contribution.

## Layer 4 — Embedded deployment (your home turf + new ML deploy)

Where your NXP work becomes the project's backbone. **Figure 2 + the DETECT/PREDICT boxes.**

| Tool            | Role                                                               | Status |
|-----------------|--------------------------------------------------------------------|--------|
| NXP eIQ Toolkit | Model optimization + deployment GUI/CLI for NXP silicon            | HAVE   |
| eIQ Auto ML SDK | End-to-end ML deploy across S32 (supports TF Lite / LiteRT + ONNX) | HAVE   |

|                                                  |                                                                           |       |
|--------------------------------------------------|---------------------------------------------------------------------------|-------|
| eIQ Neutron NPU runtime                          | The on-device accelerator in S32K5 — runs your quantized detector         | HAVE  |
| LiteRT for Microcontrollers (ex-TFLite Micro)    | The MCU inference runtime; <code>.tflite</code> flatbuffer → C byte array | LEARN |
| ONNX + ONNX Runtime                              | Interchange format between PyTorch and eIQ                                | LEARN |
| ai-edge-torch                                    | PyTorch model → <code>.tflite</code>                                      | LEARN |
| Quantization (LiteRT optimization toolkit / eIQ) | Post-training int8 — fits the model in the latency/memory budget          | LEARN |
| S32 Design Studio / MCUXpresso + S32K SDK        | Build the firmware                                                        | HAVE  |
| FreeRTOS or Zephyr                               | RTOS for the ECU runtime (Zephyr runs great in Renode)                    | LEARN |
| Lauterbach LTB / T32                             | Trace-based debug                                                         | HAVE  |
| your bare-metal / PSCI / GICv2 / MMU skills      | Real-time path correctness                                                | HAVE  |

## Layer 5 — On-vehicle real-time loop (Figure 3)

The SENSE → DETECT → PREDICT → **SUPERVISOR** → ACT loop, running on the MCU.

| Tool                                      | Role                                               | Status |
|-------------------------------------------|----------------------------------------------------|--------|
| RTOS (Zephyr / FreeRTOS / AUTOSAR)        | Schedule the loop with bounded latency             | LEARN  |
| Deterministic safety supervisor = plain C | The final veto gate. <i>Not</i> ML — your strength | HAVE   |
| vsomeip (open-source SOME/IP)             | Service-oriented comms, AUTOSAR-Adaptive-style     | OPT    |
| CycloneDDS / Fast DDS                     | Data Distribution Service for cross-ECU messaging  | OPT    |
| SocketCAN                                 | In-vehicle CAN FD                                  | LEARN  |

## Layer 6 — Off-vehicle data backbone on Railway (Figure 5)

Mostly tools you already know. **GDPR-region note in your figure is correct** — keep it EU-West.

| Tool                    | Role                                                                       | Status                            |
|-------------------------|----------------------------------------------------------------------------|-----------------------------------|
| Mosquitto / EMQX        | MQTT broker — vehicle edge → cloud                                         | LEARN                             |
| Redpanda (or Kafka)     | Ingest stream. Redpanda is lighter, Kafka-API-compatible — better for solo | LEARN                             |
| InfluxDB or TimescaleDB | Time-series telemetry store                                                | LEARN                             |
| Postgres + pgvector     | RAG memory / vector store                                                  | HAVE                              |
| FastAPI + WebSocket     | Backend + worker + live push (Python = same stack as your ML)              | HAVE -ish                         |
| Celery / arq            | Background workers (retraining, batch scoring)                             | OPT                               |
| Railway                 | Deploy everything                                                          | HAVE                              |
| Flower (flwr)           | Federated learning across vehicles                                         | OPT (optional in your own figure) |

## Layer 7 — Agent pipeline / RAG with brains (Figure 4)

| Tool                                         | Maps to        | Role                                                                  | Status    |
|----------------------------------------------|----------------|-----------------------------------------------------------------------|-----------|
| LangChain or LlamaIndex                      | pipeline glue  | RAG orchestration over pgvector                                       | LEARN     |
| Ollama (local) or API                        | Reporter (E)   | Run the LLM — your figure says <b>Qwen</b> ; Ollama runs Qwen locally | HAVE -ish |
| pgvector                                     | shared memory  | Vector store for RAG                                                  | HAVE      |
| N8N                                          | workflow glue  | Wire agents/events together visually                                  | HAVE      |
| Agents A/B/C = ML models, D = rules, E = LLM | whole pipeline | Architecture, not a single tool                                       | LEARN     |

## Layer 8 — Dashboard / UI (Figures 1 & 5)

| Tool             | Role                                                                        | Status           |
|------------------|-----------------------------------------------------------------------------|------------------|
| React + Recharts | Custom dashboard (your figure)                                              | HAVE             |
| Grafana          | Fastest path to a telemetry dashboard; pairs natively with Influx/Timescale | LEARN (easy win) |

|           |              |      |
|-----------|--------------|------|
| WebSocket | Live updates | HAVE |
|-----------|--------------|------|

**Tip:** use Grafana for the engineering/telemetry view (cheap, instant) and React+Recharts only for the bespoke "human-in-the-loop" panel that's part of your contribution. Don't rebuild Grafana in React.

## Layer 9 — DevOps / MLOps / CI

| Tool           | Role                                                   | Status |
|----------------|--------------------------------------------------------|--------|
| GitHub Actions | CI/CD (you already do MC/DC-coverage pipelines at NXP) | HAVE   |
| Docker         | Containerized services                                 | HAVE   |
| DVC            | Version datasets + models alongside code               | LEARN  |
| MLflow / W&B   | Model registry + experiment lineage                    | LEARN  |
| pytest / JUnit | Tests                                                  | HAVE   |

## Layer 10 — Safety & certification context

Not "tools" so much as methodology you must be able to speak — it's what makes a committee trust the word "self-healing."

| Item                             | Role                                                           | Status                                  |
|----------------------------------|----------------------------------------------------------------|-----------------------------------------|
| ISO 26262 / ASIL                 | Functional safety framing                                      | HAVE-ish (you touch it in HSE firmware) |
| ISO 21448 (SOTIF)                | Safety of the intended function for ML-driven behavior         | LEARN                                   |
| Safety cage / supervisor pattern | The deterministic gate over AI decisions — your Figure 6 spine | LEARN                                   |
| XAI (SHAP/Captum)                | Explainability for the audit trail                             | OPT                                     |

## Suggested install / learning order

Don't install everything at once. Follow the data:

- Layer 0** (Obsidian + Zotero) — set up the brain before you read 50 papers.
- Layer 1** (Python/uv/VS Code) — the environment.
- Layer 2 minimal** (PyTorch + scikit-learn) — learn ML on a public dataset (C-MAPSS) *first*, before touching your own data.
- Layer 3** (Renode + SocketCAN) — generate your first labeled fault dataset.
- DETECT end-to-end:** autoencoder → quantize → **Layer 4** (eIQ + LiteRT) → run on NPU. *This is your first complete vertical slice — celebrate it.*
- Layer 6** (FastAPI + Influx + pgvector on Railway) — pipe telemetry off-vehicle.
- Layer 8** (Grafana) — see it live.
- Layer 5** (supervisor + ACT) — close the loop with one recovery action.
- PREDICT** (PyG / GNN) — add cross-ECU propagation. *The novel core.*
- Layers 7 / 10 / optional** — RAG reporter, causal, RL, federated, safety case.

The honest 1-year cut: get steps 1-8 rock-solid (single fault class, single recovery action, full closed loop), then make step 9 (the GNN cross-ECU propagation) your headline contribution. Steps 10+ are "future work" in the thesis.